

Introduction à l'apprentissage par renforcement



Maxime
PLOCHARSKI

21 Novembre 2025

Plan

I. Rappels et introduction

1. MDP et équations de Bellman
2. Approches de résolution
3. Problèmes à résoudre

II. Approches Model free

1. Évaluation de politique
2. Contrôle

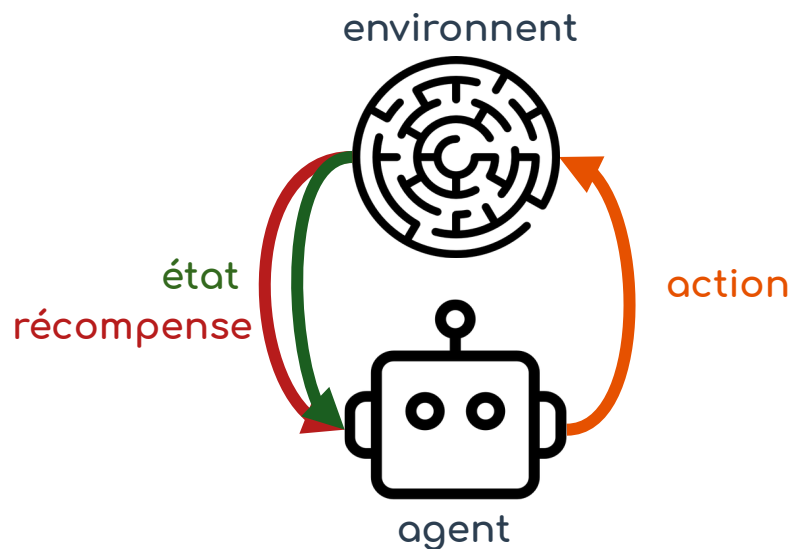
III. Apprentissage par renforcement non tabulaire

1. Principe de fonction de valeur approximée
2. Deep Q-Learning

I. Rappels et introduction

I .1 Rappel Processus décisionnel de Markov

Modélisation



MDP : $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

Équations

Fonctions de valeurs :

$$G(t) = \sum_{k=0}^{\infty} \gamma^k R(t+k)$$

$$V_{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

$$q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

Équation d'optimalité de Bellman :

$$v_{*}(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{*}(s')]$$

$$q_{*}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_{*}(s', a')]$$

I.2 Approches de résolution

Dynamique programming	Model Based	Model free
→ Connaissance parfaite du MDP	→ Estimation d'un modèle du MDP par échantillonnage	→ Apprentissage de fonction de valeur directement à partir d'échantillons
→ Planning d'une politique optimale en calculant les fonctions de valeurs	→ Permet de simplifier un MDP complexe en se restreignant à des états visités	→ Pas besoin de connaissances directe du MDP
→ Simple et déterministe mais lourd en calcul dans des cas complexes	→ Nécessite de pouvoir estimer un modèle représentatif du MDP	→ Ne converge pas nécessairement

I.3 Problèmes à résoudre

Problème de prédiction

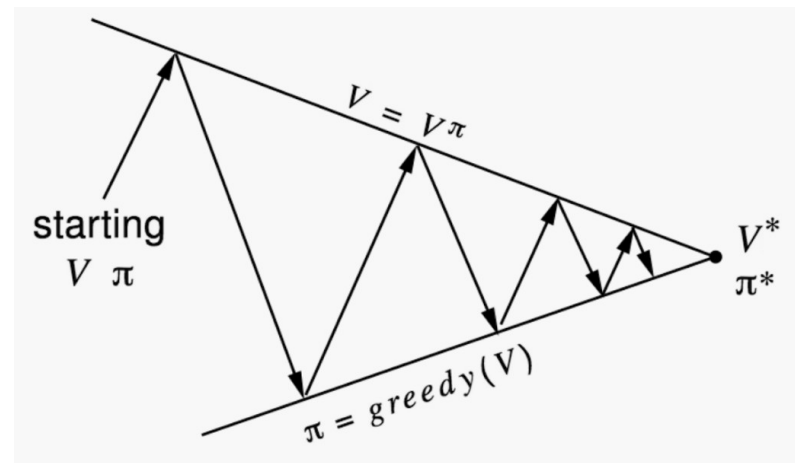
Avec un **politique donnée** estimer la fonction de valeur v_π associée

→ **Évaluation de politique**

Problème de contrôle

Trouver la **politique optimale** π_* maximisant les gains à venir, généralement en estimant la **fonction de valeur des actions** q_{π^*}

→ **Amélioration de politique**



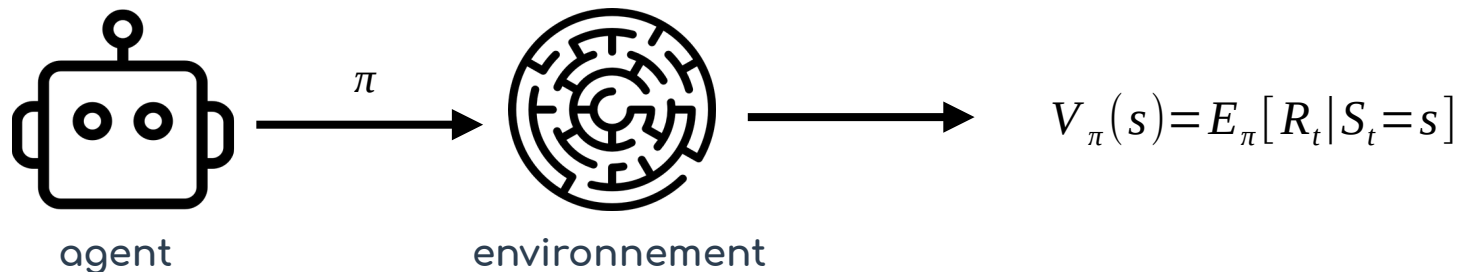
Dans le cas model free il faut **donc** **itérer entre évaluation et amélioration de la politique** jusqu'à ce qu'elle converge vers un optimum
→ **Generalized Policy Iteration (GPI)**

II. Approches Model free

II.1 Evaluation de politique

Dans un premier temps nous nous intéressons à l'estimation de notre fonctions de valeur par deux approches différentes :

- Monte Carlo
- Temporal Difference



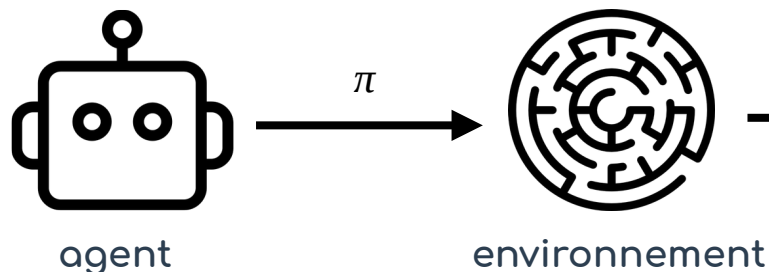
II.1 Approche Monte-Carlo

Principe générale

Problème de prédiction

Avec un politique donnée estimer la fonction de valeur v_π associée
→ **Évaluation de politique**

Pour une politique donnée π nous voulons estimer la fonction de valeur associée v_π à partir des observations de l'agent dans l'environnement en calculant **la moyenne des gains pour chaque état.**



Observations :

$s_0, s_1 r_1, \dots, s_T r_T$

$$V_\pi(s) = E_\pi[R_t | S_t = s]$$

II.1 Approche Monte-Carlo

First-visit MC policy evaluation (returns $V \approx v_\pi$)

Initialize:

$\pi \leftarrow$ policy to be evaluated

$V \leftarrow$ an arbitrary state-value function

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Repeat forever:

Generate an episode using π

For each state s appearing in the episode:

$G \leftarrow$ return following the first occurrence of s

Append G to $Returns(s)$

$V(s) \leftarrow \text{average}(Returns(s))$

II.1 Approche Monte-Carlo

Avantage :

→ Calcul empirique et direct de la fonction de valeur. Nous obtenons la fonction de valeur telle qu'elle est observée.

→ A un faible biais dans son résultat

Désavantage :

→ Nécessite beaucoup de mémoire pour stocker tout les gains et recalculer les fonctions de valeurs

→ N'est mis à jour qu'à la fin d'un épisode

→ Résultats soumis à une haute variance

→ Comment estimer la fonction de valeur de manière plus efficiente ?

II.1 TD error

Principe générale

Plus-tôt que de calculer le gain **sur tout un épisode** nous mettons à jour la fonction de valeur v_π **à chaque pas de temps** en faisant du **bootstrapping**

Équation d'optimalité de Bellman :

$$v_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')]$$

$$\rightarrow V(s_t) = R_{t+1} + \gamma V(s_{t+1})$$

$$V(s_t) = V(s_t) + \alpha * \underline{R_{t+1} + \gamma V(s_{t+1}) - V(s_t)}$$

Temporal Difference Error

II.1 Algorithm TD(0)

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

$A \leftarrow$ action given by π for S

 Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

 until S is terminal

II.1 Approche TD(0)

Avantage :

- Apprend en continu à chaque pas de temps
- A une variance faible (proportionnelle à la learning rate)

Désavantage :

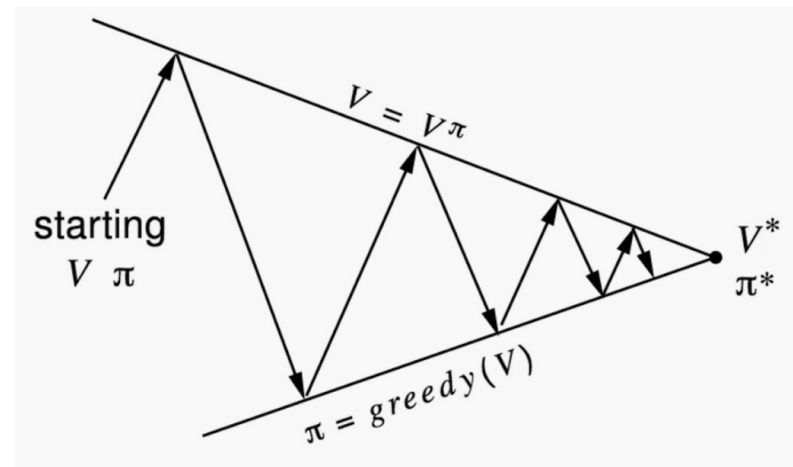
- Le bootstrapping est source d'imprécision
- Les résultats sont biaisé par le fréquence d'occurrence des états

→ Comment estimer une politique optimale ?

II.2 Contrôle model free

Le problème de contrôle se résout par le processus de GPI. Il introduit cependant un compromis à faire entre exploration et exploitation :

- Si on exploite trop la fonction de valeur estimée avec une politique glouton nous n'exploreront pas assez l'espace des états
- Plus on explore plus la convergence de l'algorithme sera longue



II.2 Contrôle model free

Pour pallier à ce problème nous pouvons utiliser lors de l'apprentissage des politiques non déterministe :

ϵ - Greedy

$$\pi(s, a) = \begin{cases} 1 - \epsilon & \text{si } a = \operatorname{argmax}_a q_\pi(s, a) \\ \epsilon / \operatorname{card}(A) - 1 & \text{si non} \end{cases}$$

Avec ϵ la probabilité de faire une action non gloutone

Boltzmann / Softmax

$$\pi(s, a) = \frac{\exp\left(\frac{q_\pi(s, a)}{\tau}\right)}{\sum_{a' \in A} \exp\left(\frac{q_\pi(s, a')}{\tau}\right)}$$

Avec τ un paramètre de température

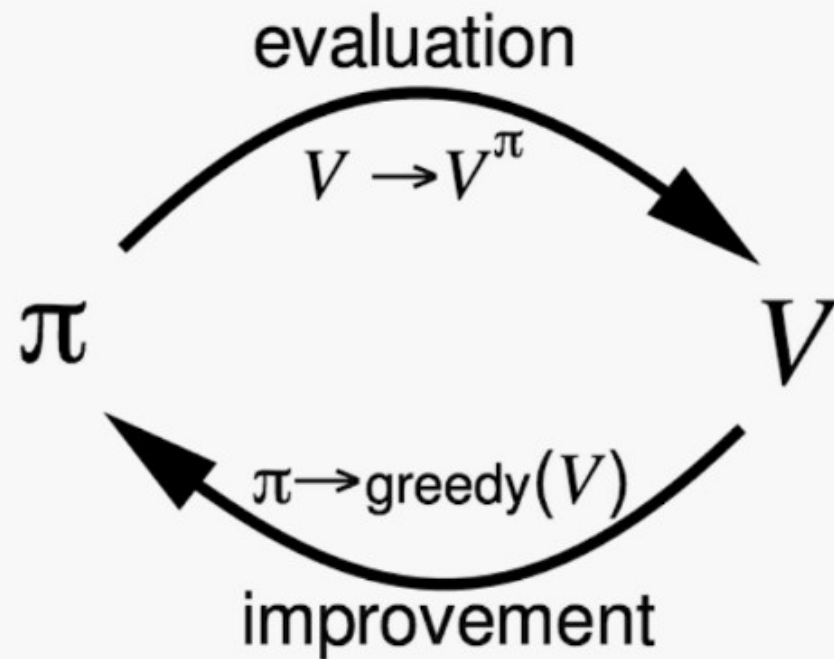
→ Cependant cela introduit de l'instabilité dans le cas d'algorithme dits « on-policy », il est donc préférable de les utiliser avec des algorithmes « off-policy »

II.1 Algorithme SARSA

Principe générale

Implémenter une GPI avec une évaluation de politique par TD.
→ State, Action, Reward, State, Action

$$q(s_t, a_t) = q(s_t, a_t) + \alpha * [R_{t+1} + \gamma q(s_{t+1}, a_{t+1}) - q(s_t, a_t)]$$



II.2 Algorithm SARSA

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$

Repeat (for each episode):

 Initialize S

 Choose A from S using policy derived from Q (e.g., ϵ -greedy)

 Repeat (for each step of episode):

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

II.2 Algorithme SARSA

Avantage :

- Apprend en continu à chaque pas de temps
- Nous livre une politique avec sa fonction de valeur associée

Désavantage :

- Compromis entre variance et biais selon le déterminisme de la politique appliquée. (epsilon faible = haut biais)

→ Comment rendre l'algorithme off-policy ?

II.1 Q-learning

Principe générale

Variante « off-policy » de SARSA permettant de stabiliser l'apprentissage.

Équation d'optimalité de Bellman :

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')]$$

$$\rightarrow q(s_t, a_t) = q(s_t, a_t) + \alpha * [R_{t+1} + \gamma \max_{a'} q(s_{t+1}, a') - q(s_t, a_t)]$$

Algo Q-learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

II.2 Algorithme Q-learning

Avantage :

- Parallélisable
- Estime directement la politique optimale
- Variance plus faible

Désavantage :

- N'estime pas directement la fonction de valeur des politiques appliquées
- A un biais plus fort et donc risque de converger vers un optimum local

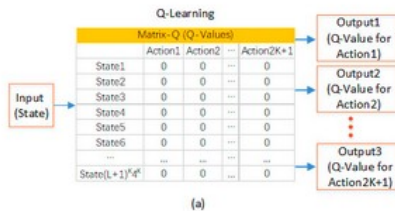
III. Apprentissage par renforcement non tabulaire

III.1 Principe de fonction de valeur approximée

$$v_{\pi}(s)$$

$\dim(\mathcal{S})$ is **big**

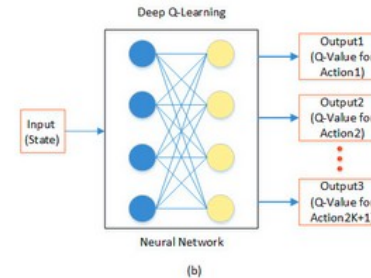
value updates are independent



$$\hat{v}_{\pi_w}(s, w)$$

$\dim(w)$ can be **much smaller**

value updates are now linked



III.1 Principe de fonction de valeur approximée

Afin d'apprendre les **fonctions de valeur approximées** nous devons ainsi définir une **fonction de loss** que l'on va minimiser par **descente de gradient**.

$$Loss: MSE = \sum_{s \in S} (v_{\pi}(s) - \hat{V}(s, w))^2 \quad \rightarrow \quad w_{t+1} = w_t - \frac{1}{2} \alpha \nabla [v_{\pi}(s_t) - \hat{V}(s_t, w_t)]$$

III.2 Deep Q-learning

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

 With probability ε select a random action a_t

 otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

 Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

 Every C steps reset $\hat{Q} = Q$

End For

End For

Références

- Article Deep-Q learning : <https://arxiv.org/pdf/1312.5602>
- *Reinforcement Learning: An Introduction*, Andrew Barto & Richard Sutton

Annexes

Monte-Carlo incrémentale

Incremental MC prediction, for estimating $V \approx v_\pi$

Input : a policy π to be evaluated

Hyperparameter : learning rate $\alpha \in (0,1]$

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily for all $s \in S$

$Returns(s) \leftarrow$ an empty list, for all $s \in S$

Loop for each episode:

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode starting from the end, $t = T - 1, T - 2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

$V(S_t) \leftarrow V(S_t) + \alpha(G - V(S_t))$

Dyna Q Algorithm for Off-Policy Control

*Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, $Q(\text{terminal state}, \cdot) = 0$,
Initialize $M(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, as an empty dictionary.*

Repeat (for each episode):

Initialize S

Repeat (for each step episode):

Choose A from S using a policy derived from Q

Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha[r + \gamma \max_a Q(S', A) - Q(S, A)]$

$M(S, A) \leftarrow R, S'$

Repeat n times:

$S_m \leftarrow \text{random previously visited state}$

$A_m \leftarrow \text{random previously taken action}$

$R_m, S'_m \leftarrow M(S_m, A_m)$

$Q(S_m, A_m) \leftarrow Q(S_m, A_m) + \alpha[R + \gamma \max_a Q(S'_m, A) - Q(S_m, A_m)]$

$S \leftarrow S'$

until S is terminal